

untangle::async

Generated by Doxygen 1.18.0

1	async	1
2	Topic Index	5
2.1	Topics	5
3	Namespace Index	7
3.1	Namespace List	7
4	Class Index	9
4.1	Class List	9
5	File Index	11
5.1	File List	11
6	Topic Documentation	13
6.1	namespace untangle: functions	13
7	Namespace Documentation	15
7.1	untangle Namespace Reference	15
7.1.1	Detailed Description	15
8	Class Documentation	17
8.1	untangle::async::attachment Struct Reference	17
8.1.1	Detailed Description	17
8.2	untangle::async::execution< actionT > Class Template Reference	17
8.2.1	Detailed Description	19
8.2.2	Constructor & Destructor Documentation	19
8.2.2.1	execution() [1/2]	19
8.2.2.2	execution() [2/2]	19
8.2.2.3	~execution()	20
8.2.3	Member Function Documentation	20
8.2.3.1	create_instance()	20
8.2.3.2	bind_action_and_method()	21
8.2.3.3	bind_action_and_function()	21
8.2.3.4	is_running()	22
8.2.3.5	is_busy()	22
8.2.3.6	run()	23
8.2.3.7	start()	23
8.2.3.8	stop()	23
8.2.3.9	add_action()	24
8.2.3.10	attach()	24
8.2.3.11	detach()	25
8.2.3.12	results()	25
8.2.4	Member Data Documentation	26
8.2.4.1	action_execute	26
8.2.4.2	action_stop	26

8.2.4.3 <code>action_is_running</code>	26
8.2.4.4 <code>on_finished</code>	26
8.2.4.5 <code>on_error</code>	27
8.3 <code>untangle::async::execution_poll</code> Class Reference	27
8.3.1 Detailed Description	28
8.3.2 Constructor & Destructor Documentation	28
8.3.2.1 <code>~execution_poll()</code>	28
8.3.3 Member Function Documentation	28
8.3.3.1 <code>add()</code>	28
8.3.3.2 <code>remove()</code>	28
8.3.3.3 <code>withdraw()</code>	29
8.3.3.4 <code>is_running()</code>	29
8.4 <code>untangle::async::invalid_attachment</code> Struct Reference	29
8.4.1 Detailed Description	30
8.4.2 Constructor & Destructor Documentation	30
8.4.2.1 <code>invalid_attachment()</code>	30
8.4.3 Member Function Documentation	30
8.4.3.1 <code>what()</code>	30
9 File Documentation	31
9.1 <code>async.hpp</code>	31
Index	39

Chapter 1

async

c++ mechanism to run queued callables in an asynchronous fashion.

The interface consists of one header file that exposes a generic execution class. It is a wrapper of a queue of callables that are invoked on other thread. The callables are pumped into the queue by invoking an action from the caller thread.

The implementation is using the `actuator` callable, and therefore it exemplifies how a generic callable may improve the code structure, by easily creating interfaces inside a class instead of using external ones.

There are provided two execution modes: one-off and continuous.

requirements

C++23. The interface reports warnings with `std::println`, so it needs a standard library that provides `<print>` — GCC 14 or newer; older `libc++` and MSVC releases may not have it yet.

one-off execution



example: one-off execution of a plain function

```
#include <async.hpp>

void f()
{
    std::cout << "f thread " << std::this_thread::get_id() << std::endl;
}

int main()
{
    std::cout << "main thread " << std::this_thread::get_id() << std::endl;

    // declare an execution specialized for an action type; it must be held by a shared pointer,
    // because an async binding keeps only weak ownership of it
    // the name is optional, and identifies the execution in the warnings it reports
    using void_execution = untangle::async::execution<std::function<void(void)>;
    auto execution = void_execution::create_instance("oneoff");
```

```

std::function<void(void)> action;
// create async binding between the action and f; the execution is passed as a shared pointer,
// which the binding holds weakly - that is what lets the action outlive it safely
void_execution::bind_action_and_function(action, f, execution);
// whenever the action is invoked, a new callable wrapping f will be added to the execution's action queue
action();
// run the execution
execution->run();

// add the execution object to a poll of one's own; the poll answers for what was added to it
untangle::async::execution_poll poll;
poll.add(*execution);

// wait the polled executions to finish
while(poll.is_running())
{
    std::this_thread::sleep_for(std::chrono::milliseconds(1));
}

return 0;
}

```

example: one-off execution of a class method

```

#include <async.hpp>

class A
{
public:
    void f(int x)
    {
        std::cout << "A::f " << x << " thread " << std::this_thread::get_id() << std::endl;
    }
    std::function<void(int)> action;
};

int main()
{
    std::cout << "main thread " << std::this_thread::get_id() << std::endl;

    auto a = std::make_shared<A>();

    // declare an execution specialized for an action type; it must be held by a shared pointer
    using int_execution = untangle::async::execution<std::function<void(int)>>;
    auto execution = int_execution::create_instance("oneoff_method");
    // note both the bound object and the execution must be shared pointers
    int_execution::bind_action_and_method(a->action, a, &A::f, execution);
    // whenever the action is invoked, a new callable wrapping f will be added to the execution's action queue
    a->action(10);
    // run the execution
    execution->run();

    // add the execution object to a poll of one's own; the poll answers for what was added to it
    untangle::async::execution_poll poll;
    poll.add(*execution);

    // wait the polled executions to finish
    while(poll.is_running())
    {
        std::this_thread::sleep_for(std::chrono::milliseconds(1));
    }

    return 0;
}

```

continuous execution



example: continuous execution of a plain function

```

#include <async.hpp>

void f()
{

```

```

    std::cout << "f thread " << std::this_thread::get_id() << std::endl;
}

int main()
{
    std::cout << "main thread " << std::this_thread::get_id() << std::endl;

    // declare an execution specialized for an action type; it must be held by a shared pointer
    using void_execution = untangle::async::execution<std::function<void(void)>>;
    auto execution = void_execution::create_instance("continuous");
    std::function<void(void)> action;
    // create async binding between the action and f
    void_execution::bind_action_and_function(action, f, execution);

    // run the execution
    execution->start();
    // add f to the execution's queue (three times)
    action();action();action();
    // stop the execution
    execution->stop();

    // add the execution object to a poll of one's own; the poll answers for what was added to it
    untangle::async::execution_poll poll;
    poll.add(*execution);

    // wait the polled executions to finish
    while(poll.is_running())
    {
        std::this_thread::sleep_for(std::chrono::milliseconds(1));
    }

    return 0;
}

```

queueing and lifetime

Queueing is thread safe. An action may be invoked from any thread; the execution's list is guarded, and the worker is woken rather than polled for.

`stop()` ends the execution's working life. The worker runs whatever was queued before the call and then leaves. An action invoked after `stop()` is refused rather than queued, because it would never run, and the refusal is reported:

```
warning: execution 'continuous' is stopped, action not added
```

An action bound to an object that has since been destroyed is dropped the same way, rather than ending the process:

```
warning: execution 'continuous' dropped an invalid action: bind: invalid object
```

The worker is detached and cannot be joined. Waiting for an execution therefore means waiting on its running state, which is what `execution_poll` reports and what the examples above do. The destructor waits on the same state, so an execution that goes out of scope while its worker is still running blocks until the worker is finished rather than leaving it reading freed memory.

The poll holds a pointer to each execution added to it, so an execution withdraws itself in its destructor - from every poll it was added to, since it may be in more than one. A poll destroyed first likewise releases what it still holds, so a poll may be a local. `execution_poll::remove()` is available for withdrawing one earlier; calling it is not required.

building the tests

```

cmake -S test -B test/build
cmake --build test/build
ctest --test-dir test/build

```

`test/CMakeLists.txt` fetches googletest at configure time, so the first configure needs network access. The actuator submodule has to be present: `git submodule update --init`.

sanitizers

ASYNC_SANITIZE builds the tests under a sanitizer. It is off by default, because a sanitized build is several times slower and ThreadSanitizer does not ship for every toolchain.

```
cmake -S test -B test/build-asan -DASYNC_SANITIZE=address
cmake --build test/build-asan
ctest --test-dir test/build-asan
```

Accepted values are address, thread, undefined, or empty. Anything else is refused at configure time rather than passed through to the compiler.

Use a separate build directory per sanitizer. address and thread instrument the same accesses in incompatible ways and cannot be combined, so one build directory is one sanitizer.

The two answer different questions. ThreadSanitizer names both sides of a data race, which is how the queue's races were diagnosed. AddressSanitizer reports a use-after-free, which is how the dangling attachment was. Neither sees everything: a defect reached through a destroyed `std::mutex` throws `std::system_error` from inside `libsystem` before any instrumented load runs, so both sanitizers stay silent. Reach for a direct instrumented read when that happens.

Chapter 2

Topic Index

2.1 Topics

Here is a list of all topics with brief descriptions:

namespace untangle: functions [13](#)

Chapter 3

Namespace Index

3.1 Namespace List

Here is a list of all documented namespaces with brief descriptions:

untangle	Interface to untangle::async::execution class	15
--------------------------	---	--------------------

Chapter 4

Class Index

4.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

untangle::async::attachment	
One execution's place in the attachment graph: who attached it, if anyone	17
untangle::async::execution< actionT >	
Asynchronous template execution class	17
untangle::async::execution_poll	
Poll that reports whether any execution added to it is still running	27
untangle::async::invalid_attachment	
Raised by execution::attach() when the attachment asked for is refused	29

Chapter 5

File Index

5.1 File List

Here is a list of all documented files with brief descriptions:

async.hpp		31
---------------------------	-------	----

Chapter 6

Topic Documentation

6.1 namespace untangle: functions

Chapter 7

Namespace Documentation

7.1 untangle Namespace Reference

Interface to [untangle::async::execution](#) class.

7.1.1 Detailed Description

Interface to [untangle::async::execution](#) class.

Chapter 8

Class Documentation

8.1 `untangle::async::attachment` Struct Reference

One execution's place in the attachment graph: who attached it, if anyone.

```
#include <async.hpp>
```

Public Attributes

- `std::weak_ptr< attachment >` `attacher`
The link of the execution that attached this one; empty when nothing has.

8.1.1 Detailed Description

One execution's place in the attachment graph: who attached it, if anyone.

At namespace scope, so it is one type rather than one per specialisation: an execution may be attached by an execution of any specialisation, and the chain is walked without knowing any link's action type. It dies with its execution, so it doubles as a liveness token - a link that still locks belongs to an execution that is still there.

The documentation for this struct was generated from the following file:

- `async.hpp`

8.2 `untangle::async::execution< actionT >` Class Template Reference

Asynchronous template execution class.

```
#include <async.hpp>
```

Public Member Functions

- `execution ()`
Constructs an execution with the default name. Call `create_instance()` instead.
- `execution (std::string exec_name)`
Constructs a named execution. Use `create_instance()` for one that is to be bound.
- `~execution ()`
Destroys the execution, once its worker has left.
- `bool is_running () const`
Is a worker of this execution's still there?
- `bool is_busy () const`
Is this execution working through its action queue?
- `void run ()`
Runs what is queued on a worker of its own, once, and keeps the results.
- `void start ()`
Starts a continuous worker that runs whatever is queued until `stop()`.
- `void stop ()`
Stops the worker, after the actions already queued have run.
- `template<typename... Args>`
`bool add_action (actionT action, Args &&... args)`
Queues an action, bound to args, and says whether it was taken.
- `template<typename otherT>`
`void attach (const std::shared_ptr< otherT > &other)`
Attaches another execution, so that this one triggers it.
- `template<typename otherT>`
`bool detach (otherT &other)`
Detaches an execution previously attached by `attach()`.
- `auto results () const`
The return values of the actions run by `run()`, in the order they ran.

Static Public Member Functions

- `template<typename... Args>`
`static std::shared_ptr< execution > create_instance (Args &&... args)`
Creates an execution owned by a `std::shared_ptr`, which is what binding to it requires.
- `template<typename classT, typename T>`
`static void bind_action_and_method (actionT &action, const std::shared_ptr< classT > &obj, T classT::*method, const std::shared_ptr< execution > &async_exec)`
Binds asynchronously an external action to a class function member.
- `template<typename T>`
`static void bind_action_and_function (actionT &action, T Fn, const std::shared_ptr< execution > &async_exec)`
Binds asynchronously an external action to a plain function.

Public Attributes

- `std::function< void(void)> action_execute`
Connection point that runs this execution's queued actions. Bound to `execute_actions()`.
- `std::function< void(void)> action_stop`
Connection point that stops this execution. Bound to `stop()`.
- `std::function< bool(void)> action_is_running`
Connection point that reports whether this execution is running. Bound to `is_running()`.
- `std::function< void(void)> on_finished`
Called on the worker's thread once the action queue has drained. Assigned by the caller.
- `std::function< void(std::exception_ptr)> on_error`
Called on the worker's thread with whatever an action threw. Assigned by the caller.

Static Public Attributes

- `static constexpr auto default_name = "default_name"`
Name of an execution built unnamed.

8.2.1 Detailed Description

```
template<typename actionT>
class untangle::async::execution< actionT >
```

Asynchronous template execution class.

An async execution provides a mechanism to queue actions and execute them sequentially on a separate thread. It may also attach another [execution](#) object and trigger its actions. This way actions of different types may be executed on the same thread.

The mechanism relies on an "asynchronous binding" created by [bind_action_and_method\(\)](#) or [bind_action_and_function\(\)](#).

Template Parameters

<code>actionT</code>	It represents the type of the action. It is specified as <code>std::function<...></code> and should match the signature of the bound function or class method.
----------------------	--

8.2.2 Constructor & Destructor Documentation

8.2.2.1 `execution()` [1/2]

```
template<typename actionT>
untangle::async::execution< actionT >::execution () [inline]
```

Constructs an execution with the default name. Call [create_instance\(\)](#) instead.

The name identifies this execution in the warnings it reports, and is otherwise unused.

8.2.2.2 `execution()` [2/2]

```
template<typename actionT>
untangle::async::execution< actionT >::execution (
    std::string exec_name) [inline], [explicit]
```

Constructs a named execution. Use [create_instance\(\)](#) for one that is to be bound.

Parameters

exec_name	- A name for this execution.
-----------	------------------------------

8.2.2.3 ~execution()

```
template<typename actionT>
untangle::async::execution< actionT >::~~execution () [inline]
```

Destroys the execution, once its worker has left.

The worker is detached and cannot be joined, so this waits on the "running" state instead. It detaches this execution from whatever attached it first, and returns at once for one that was never started.

Attention

Safe only because running_ is cleared as the very last thing the worker does: this object may be freed the moment it reads false, so nothing may follow it in execute() or loop().

Remarks

No rule of five, deliberately. An execution is declared once and used in place; the members already delete the copy operations several times over.

8.2.3 Member Function Documentation

8.2.3.1 create_instance()

```
template<typename actionT>
template<typename... Args>
std::shared_ptr< execution > untangle::async::execution< actionT >::create_instance (
    Args &&... args) [inline], [static]
```

Creates an execution owned by a std::shared_ptr, which is what binding to it requires.

An execution built any other way is still usable - `add_action()` takes actions directly - but cannot be bound, and the attempt does not compile.

Parameters

args	- Constructor arguments: a name, or nothing for the default name.
------	---

Returns

- A std::shared_ptr owning the new execution.

8.2.3.2 bind_action_and_method()

```
template<typename actionT>
template<typename classT, typename T>
void untangle::async::execution< actionT >::bind_action_and_method (
    actionT & action,
    const std::shared_ptr< classT > & obj,
    T classT::* method,
    const std::shared_ptr< execution< actionT > > & async_exec) [inline], [static]
```

Binds asynchronously an external action to a class function member.

It creates an action as a binding to a class method (by untangle::bind()), and assigns to action a callable that queues it. The execution is held weakly, so the action may outlive it; the action itself is shared with every call queued from it, rather than copied per call.

Attention

Invoking action after the execution has been destroyed throws untangle::invalid_action. The worker catches it and drops the action with a warning.

Parameters

action	[in,out] - An action of type std::function<...>.
obj	- A std::shared_ptr that wraps the bound class object.
method	- Pointer to function member. It is specified as &<class type>::<function member>.
async_exec	- A std::shared_ptr owning the execution the action is queued on. Requiring it here is what keeps an execution that no shared_ptr owns from being bound at all.

8.2.3.3 bind_action_and_function()

```
template<typename actionT>
template<typename T>
void untangle::async::execution< actionT >::bind_action_and_function (
    actionT & action,
    T Fn,
    const std::shared_ptr< execution< actionT > > & async_exec) [inline], [static]
```

Binds asynchronously an external action to a plain function.

It wraps Fn in an action and assigns to action a callable that queues it. The execution is held weakly, as in [bind_action_and_method\(\)](#), and the action is shared with every call queued from it rather than copied per call.

Parameters

action	[in,out] - An action of type std::function<...>.
Fn	- A plain function, or any callable; moved into the binding.

async_exec	- A std::shared_ptr owning the execution the action is queued on.
------------	---

8.2.3.4 is_running()

```
template<typename actionT>
bool untangle::async::execution< actionT >::is_running () const [inline]
```

Is a worker of this execution's still there?

True from `run()` or `start()` until the worker has left, which is the same instant `~execution()` waits for and the same one `run()` waits for before starting another.

Remarks

`on_finished` is raised from inside the drain and reads this as true - the worker is standing in the callback with the rest of its path still to go.

Returns

true - a worker is still there.

8.2.3.5 is_busy()

```
template<typename actionT>
bool untangle::async::execution< actionT >::is_busy () const [inline]
```

Is this execution working through its action queue?

True from the moment an action is added until the last one has returned. Unlike `is_running()`, which is about the worker thread, this answers "has it room for more work"

- a worker started by `start()` is running for its whole life whether busy or idle.

Remarks

Answers for this execution's own actions, not for those it triggers through `attach()`.

A false answer is durable only for the caller that is itself adding the actions; a true one may stop being true the moment after.

Returns

true - actions are queued, or one is running.

8.2.3.6 run()

```
template<typename actionT>
void untangle::async::execution< actionT >::run () [inline]
```

Runs what is queued on a worker of its own, once, and keeps the results.

The worker is detached: wait for it with `is_running()` or `execution_poll`. Actions queued while it is draining are run by it too. `results()` is filled by this path only.

Blocks until a worker still resident from a previous run has left, so one execution never has two. `is_running()` reads false at exactly that moment, so a caller that waits for it and then runs again does not wait here at all.

Attention

Never call this from inside an action: the worker that would have to leave is the one making the call, so it waits for itself and hangs.

8.2.3.7 start()

```
template<typename actionT>
void untangle::async::execution< actionT >::start () [inline]
```

Starts a continuous worker that runs whatever is queued until `stop()`.

Unlike `run()`, the worker stays for the life of the execution, so `is_running()` is true whether or not there is work; ask `is_busy()` instead. Results are not collected.

Blocks until a worker still resident has left, as `run()` does.

Attention

Never call this from inside an action, for the reason given on `run()`.

8.2.3.8 stop()

```
template<typename actionT>
void untangle::async::execution< actionT >::stop () [inline]
```

Stops the worker, after the actions already queued have run.

Remarks

An action passed to `add_action()` after this call is refused, not queued: `stop()` is the end of this execution's working life, and an action accepted then would never run.

8.2.3.9 add_action()

```
template<typename actionT>
template<typename... Args>
bool untangle::async::execution< actionT >::add_action (
    actionT action,
    Args &&... args) [inline]
```

Queues an action, bound to args, and says whether it was taken.

It runs later, on whichever thread drives this execution - its own worker, or an attacher's. Queueing before [run\(\)](#) or [start\(\)](#) is normal.

Attention

A stopped execution refuses and drops the action, silently for a caller that ignores the answer. Nothing is thrown: `untangle::invalid_action` means a dead binding, while a refused action is sound and merely too late.

Remarks

A caller arriving through [bind_action_and_method\(\)](#) or [bind_action_and_function\(\)](#) never sees the answer: those lambdas return `actionT::result_type`, which has no room for it.

Template Parameters

Args	- The argument types the action is bound to.
------	--

Parameters

action	- The action to queue.
args	- The arguments to bind to action.

Returns

true - queued, and this execution's worker will run it.
false - the execution is stopped and the action was dropped.

8.2.3.10 attach()

```
template<typename actionT>
template<typename otherT>
void untangle::async::execution< actionT >::attach (
    const std::shared_ptr< otherT > & other) [inline]
```

Attaches another execution, so that this one triggers it.

The attached execution's pending actions are run by this one's worker, and stopping this one stops that one too. Actions of different types can be run on one thread this way.

Parameters

other	- A std::shared_ptr owning the execution to trigger; requiring one keeps an unowned execution from being attached at all. Attaching the same execution twice adds it twice, and one detach() still removes it completely.
-------	---

Exceptions

invalid_attachment	- if other is attached already. An execution has at most one attacher, which keeps the graph a forest and the cycle check a walk rather than a search.
invalid_attachment	- if the attachment would close a cycle, itself included. Driving a cycle recurses until the stack is gone, so it is refused where the caller can still be told.

8.2.3.11 detach()

```
template<typename actionT>
template<typename otherT>
bool untangle::async::execution< actionT >::detach (
    otherT & other) [inline]
```

Detaches an execution previously attached by attach().

Both paths are unwired together, the triggering one and the stopping one.

Parameters

other	- The execution to stop triggering. Detaching one that was never attached is not an error: the caller asked for a state that already holds.
-------	---

Returns

true - other was attached and is no longer.
false - other was not attached to this execution.

8.2.3.12 results()

```
template<typename actionT>
auto untangle::async::execution< actionT >::results () const [inline]
```

The return values of the actions run by run(), in the order they ran.

Read it from on_finished, which fires once the queue has drained, so the results are complete by the time the callback sees them. Returned by value: the worker is appending to it.

Remarks

Only run() fills this. A continuous worker has no point at which a run is over, so start() collects nothing.

Returns

The results of the most recent run, or empty if none has produced any.

8.2.4 Member Data Documentation

8.2.4.1 `action_execute`

```
template<typename actionT>
std::function<void(void)> untangle::async::execution< actionT >::action_execute
```

Connection point that runs this execution's queued actions. Bound to `execute_actions()`.

Attention

Wired by the constructor. Assigning to it unwires whatever is connected, silently.

8.2.4.2 `action_stop`

```
template<typename actionT>
std::function<void(void)> untangle::async::execution< actionT >::action_stop
```

Connection point that stops this execution. Bound to `stop()`.

`attach()` wires it together with `action_execute`, and `detach()` unwires both.

Attention

Wired by the constructor. Assigning to it unwires whatever is connected, silently.

8.2.4.3 `action_is_running`

```
template<typename actionT>
std::function<bool(void)> untangle::async::execution< actionT >::action_is_running
```

Connection point that reports whether this execution is running. Bound to `is_running()`.

`execution_poll` holds its address, and `~execution()` takes it back out.

Attention

Wired by the constructor. Assigning to it unwires whatever is connected, silently.

8.2.4.4 `on_finished`

```
template<typename actionT>
std::function<void(void)> untangle::async::execution< actionT >::on_finished
```

Called on the worker's thread once the action queue has drained. Assigned by the caller.

`results()` is complete by the time it runs, and `is_running()` reads true there.

8.2.4.5 on_error

```
template<typename actionT>
std::function<void(std::exception_ptr)> untangle::async::execution< actionT >::on_error
```

Called on the worker's thread with whatever an action threw. Assigned by the caller.

The action is dropped either way; this is what it is dropped to. Without it the only record is a warning on stderr naming this execution, which reaches a log and no code — an action that failed and one that succeeded are the same from outside. A handler assigned here replaces that warning, so a caller that takes responsibility for reporting is not reported twice.

Remarks

It receives a `std::exception_ptr` because the worker catches everything, including what is not a `std::exception`. Rethrow it to read it.

It is told about a dead binding too, which arrives as `untangle::invalid_action`. A caller asking why an action did not run wants both answers, not the one the header happens to raise itself.

Attention

It runs on the worker's thread, inside the drain, like `on_finished`. Nothing may escape it: it is called from the same try the action was called from, and an exception leaving a detached thread function calls `std::terminate`.

The documentation for this class was generated from the following file:

- `async.hpp`

8.3 untangle::async::execution_poll Class Reference

Poll that reports whether any `execution` added to it is still running.

```
#include <async.hpp>
```

Public Member Functions

- `execution_poll ()=default`
Constructs an empty poll. `add()` is what puts anything in it.
- `~execution_poll ()`
Destroys the poll, releasing every execution still in it.
- `template<typename asyncexecT>`
`void add (asyncexecT &async_exec)`
Adds an `execution` object to the poll.
- `template<typename asyncexecT>`
`void remove (asyncexecT &async_exec)`
Removes an `execution` object from the poll.
- `void withdraw (const std::function< bool(void)> *action)`
Takes one action pointer out, without touching the execution it came from.
- `auto is_running ()`
Checks if the execution objects are running.

8.3.1 Detailed Description

Poll that reports whether any [execution](#) added to it is still running.

Remarks

An execution runs in a detached thread and cannot be joined, so checking the "running" state is the only way to wait for one. This checks any number of them at once.

A poll is constructed and owned by the caller, and answers for the executions added to that one. An owner of a set of workers - a thread pool is the case this was written for - waits for its own and not for whatever else the process is running.

8.3.2 Constructor & Destructor Documentation

8.3.2.1 ~execution_poll()

```
untangle::async::execution_poll::~execution_poll () [inline]
```

Destroys the poll, releasing every execution still in it.

Remarks

The record runs both ways, and it has to: each side holds a pointer into the other, so whichever dies first takes itself out of the other. ~execution() withdraws from every poll holding it; this tells every execution it holds that this poll is gone. Neither order leaves a dangling pointer, which is what lets a poll be a local.

8.3.3 Member Function Documentation

8.3.3.1 add()

```
template<typename asyncexecT>
void untangle::async::execution_poll::add (
    asyncexecT & async_exec) [inline]
```

Adds an [execution](#) object to the poll.

Parameters

async_exec	An execution object.
------------	--------------------------------------

8.3.3.2 remove()

```
template<typename asyncexecT>
void untangle::async::execution_poll::remove (
    asyncexecT & async_exec) [inline]
```

Removes an [execution](#) object from the poll.

Attention

Parameters

async_exec	An execution object.
------------	--------------------------------------

8.3.3.3 withdraw()

```
void untangle::async::execution_poll::withdraw (
    const std::function< bool(void)> * action) [inline]
```

Takes one action pointer out, without touching the execution it came from.

What `~execution()` calls for each poll holding it: by then the object is being destroyed and must not be called back into, so the two halves of [remove\(\)](#) are separated here.

Parameters

action	- The address add() stored.
--------	---

8.3.3.4 is_running()

```
auto untangle::async::execution_poll::is_running () [inline]
```

Checks if the execution objects are running.

Returns

true - if at least one [execution](#) object in this poll is running.

The documentation for this class was generated from the following file:

- `async.hpp`

8.4 untangle::async::invalid_attachment Struct Reference

Raised by [execution::attach\(\)](#) when the attachment asked for is refused.

```
#include <async.hpp>
```

Public Member Functions

- [invalid_attachment](#) (std::string text)
Construct a new invalid attachment object.
- const char * [what](#) () const noexcept override
The message text describing the reason of this exception.

8.4.1 Detailed Description

Raised by `execution::attach()` when the attachment asked for is refused.

Remarks

Not an `untangle::invalid_action`: that one reports a binding whose target has died and is swallowed by the worker, while this is a caller error and must not be.

8.4.2 Constructor & Destructor Documentation

8.4.2.1 `invalid_attachment()`

```
untangle::async::invalid_attachment::invalid_attachment (
    std::string text) [inline], [explicit]
```

Construct a new invalid attachment object.

Parameters

text	- A message text, describing the reason of this exception.
------	--

8.4.3 Member Function Documentation

8.4.3.1 `what()`

```
const char * untangle::async::invalid_attachment::what () const [inline], [override], [noexcept]
```

The message text describing the reason of this exception.

Returns

const char* - The message text.

The documentation for this struct was generated from the following file:

- `async.hpp`

Chapter 9

File Documentation

9.1 async.hpp

```
00001 // Copyright (c) 2018 Nicolae Popescu. MIT License.
00002
00006 #pragma once
00007
00008 #include <actuator/actuator.hpp>
00009 #include <algorithm>
00010 #include <atomic>
00011 #include <chrono>
00012 #include <condition_variable>
00013 #include <deque>
00014 #include <exception>
00015 #include <functional>
00016 #include <memory>
00017 #include <mutex>
00018 #include <print>
00019 #include <string>
00020 #include <thread>
00021 #include <type_traits>
00022 #include <utility>
00023 #include <vector>
00024
00025 namespace untangle {
00026 namespace async {
00030
00037 struct invalid_attachment : std::exception {
00043     explicit invalid_attachment(std::string text) : message_(std::move(text)) {}
00044
00050     const char* what() const noexcept override { return message_.c_str(); }
00051
00052 private:
00053     std::string message_;
00054 };
00055
00064 struct attachment {
00066     std::weak_ptr<attachment> attacher;
00067 };
00068
00079 class execution_poll {
00080 public:
00084     execution_poll() = default;
00085
00094     ~execution_poll() {
00095         std::vector<registration> held;
00096
00097         {
00098             std::lock_guard<std::mutex> lock(actuator_mutex_);
00099             held.swap(registrations_);
00100         }
00101
00102         // Outside the lock: each of these takes the execution's own, and ~execution() takes them in
00103         // the opposite order. Releasing this one first is what keeps the two from meeting in a cycle.
00104         for (auto& one : held) {
00105             one.forget();
00106         }
00107     }
00108
00114     template <typename asyncexecT>
```

```

00115 void add(asyncexecT& async_exec) {
00116     {
00117         std::lock_guard<std::mutex> lock(actuator_mutex_);
00118
00119         if (!actuator_is_running_.is_connected()) {
00120             actuator_is_running_ = untangle::connect(async_exec.action_is_running);
00121         } else {
00122             actuator_is_running_.add(&async_exec.action_is_running);
00123         }
00124
00125         registrations_.emplace_back(&async_exec.action_is_running,
00126                                     [this, &async_exec] { async_exec.forget_poll(*this); });
00127     }
00128
00129     // Outside the lock, and deliberately: this takes the execution's own. The execution has to
00130     // know which polls hold a pointer into it, because ~execution() is what takes it back out and
00131     // there is now more than one poll it could be in.
00132     async_exec.remember_poll(*this);
00133 }
00134
00143 template <typename asyncexecT>
00144 void remove(asyncexecT& async_exec) {
00145     withdraw(&async_exec.action_is_running);
00146     async_exec.forget_poll(*this);
00147 }
00148
00157 void withdraw(const std::function<bool(void)>* action) {
00158     std::lock_guard<std::mutex> lock(actuator_mutex_);
00159
00160     actuator_is_running_.remove(action);
00161     std::erase_if(registrations_,
00162                  [action](const registration& one) { return one.action == action; });
00163 }
00164
00170 auto is_running() {
00171     std::lock_guard<std::mutex> lock(actuator_mutex_);
00172     actuator_is_running_();
00173
00174     auto result = false;
00175     for (const auto& ret : actuator_is_running_.results) {
00176         result = result || ret;
00177     }
00178     return result;
00179 }
00180
00181 private:
00186 struct registration {
00187     const std::function<bool(void)>* action;
00188     std::function<void()> forget;
00189 };
00190
00191 actuator<std::function<bool(void)>> actuator_is_running_;
00192 std::vector<registration> registrations_;
00193 mutable std::mutex actuator_mutex_;
00194 };
00195
00209 template <typename actionT>
00210 class execution {
00211     // An execution may attach one of a different specialisation, which is a different class;
00212     // attach() and detach() reach into its attachment_lifetime_.
00213     template <typename otherActionT>
00214     friend class execution;
00215
00216     // add() and remove() keep the record this object holds of the polls that hold it.
00217     friend class execution_poll;
00218
00219 public:
00229     template <typename... Args>
00230     static std::shared_ptr<execution> create_instance(Args&&... args) {
00231         return std::make_shared<execution>(std::forward<Args>(args)...);
00232     }
00233
00239     execution() : execution(std::string(default_name)) {}
00240
00246     explicit execution(std::string exec_name) : name(std::move(exec_name)) {
00247         action_execute = untangle::bind(this, &execution<actionT>::execute_actions);
00248         action_stop = untangle::bind(this, &execution<actionT>::stop);
00249         action_is_running = untangle::bind(this, &execution<actionT>::is_running);
00250     }
00251
00266     ~execution() {
00267         // Out of the attacher first, before the worker is even asked to stop: from here on nothing
00268         // may reach this object, and the actions about to be destroyed are the ones it holds.
00269         if (!attachment_lifetime_->attacher.expired()) {
00270             attacher_execute->remove(&action_execute);
00271             attacher_stop->remove(&action_stop);
00272         }

```

```

00273
00274 {
00275     std::lock_guard<std::mutex> lock(action_mutex_);
00276     started_ = false;
00277     stopped_ = true;
00278 }
00279
00280 action_cv_.notify_all();
00281
00282 while (running_.load()) { // time of check
00283     std::this_thread::sleep_for(std::chrono::milliseconds(1));
00284 }
00285
00286 // time of use: this object is freed once the destructor returns, so the check above is only
00287 // safe because running_ is the last thing the worker touches.
00288 //
00289 // Out of every poll that holds a pointer into this object: an execution can be in several,
00290 // and an address left in any of them outlives the object. Taken by swap so nothing else can be
00291 // added to the record while this walks it, and withdrawn rather than removed because remove()
00292 // would call back into an object that is going away.
00293 std::vector<execution_poll*> holding_polls;
00294
00295 {
00296     std::lock_guard<std::mutex> lock(polls_mutex_);
00297     holding_polls.swap(polls_);
00298 }
00299
00300 for (auto* poll : holding_polls) {
00301     poll->withdraw(&action_is_running);
00302 }
00303 }
00304
00322 template <typename classT, typename T>
00323 static void bind_action_and_method(actionT& action, const std::shared_ptr<classT>& obj,
00324                                     T classT::* method,
00325                                     const std::shared_ptr<execution>& async_exec) {
00326     auto async_action = std::make_shared<const actionT>(untangle::bind(obj, method));
00327
00328     action = [wp = std::weak_ptr<execution>(async_exec),
00329               async_action](auto&&... args) -> actionT::result_type {
00330         // lock() also keeps the execution alive for the duration of the call
00331         const auto exec = wp.lock();
00332         if (!exec) {
00333             throw invalid_action("bind: invalid execution");
00334         }
00335
00336         exec->add_queued_action([async_action, ... args = std::forward<decltype(args)>(args)] {
00337             return (*async_action)(args...);
00338         });
00339
00340         return typename actionT::result_type();
00341     };
00342 }
00343
00355 template <typename T>
00356 static void bind_action_and_function(actionT& action, T Fn,
00357                                     const std::shared_ptr<execution>& async_exec) {
00358     auto async_action = std::make_shared<const actionT>(std::move(Fn));
00359
00360     action = [wp = std::weak_ptr<execution>(async_exec),
00361               async_action](auto&&... args) -> actionT::result_type {
00362         const auto exec = wp.lock();
00363         if (!exec) {
00364             throw invalid_action("bind: invalid execution");
00365         }
00366
00367         exec->add_queued_action([async_action, ... args = std::forward<decltype(args)>(args)] {
00368             return (*async_action)(args...);
00369         });
00370
00371         return typename actionT::result_type();
00372     };
00373 }
00374
00386 bool is_running() const { return running_.load(); }
00387
00403 bool is_busy() const {
00404     std::lock_guard<std::mutex> lock(action_mutex_);
00405     return !action_queue_.empty() || executing_action_.load();
00406 }
00407
00421 void run() {
00422     wait_thread_to_finish();
00423     collecting_results_ = true;
00424     thread_ = std::thread(&execution::execute, this);
00425     thread_.detach();
00426 }

```

```

00427
00438 void start() {
00439     wait_thread_to_finish();
00440     collecting_results_ = false;
00441     {
00442         std::lock_guard<std::mutex> lock(action_mutex_);
00443         started_ = true;
00444         stopped_ = false;
00445     }
00446     thread_ = std::thread(&execution::loop, this);
00448     thread_.detach();
00449 }
00450
00457 void stop() {
00458     {
00459         std::lock_guard<std::mutex> lock(action_mutex_);
00460         started_ = false;
00461         stopped_ = true;
00462     }
00463
00464     // The worker drains what is already queued before it leaves loop(), so stop() does not have to
00465     // spin on the queue; waking it is enough.
00466     action_cv_.notify_all();
00467
00468     if (actuator_stop_.is_connected()) {
00469         actuator_stop_();
00470     }
00471 }
00472
00494 template <typename... Args>
00495 bool add_action(actionT action, Args&&... args) {
00496     return add_queued_action(std::bind(std::move(action), std::forward<Args>(args)...));
00497 }
00498
00518 template <typename otherT>
00519 void attach(const std::shared_ptr<otherT>& other) {
00520     if (static_cast<const void*>(other.get()) == static_cast<const void*>(this)) {
00521         throw invalid_attachment("attach: an execution cannot be attached to itself");
00522     }
00523
00524     if (!other->attachment_lifetime_->attacher.expired()) {
00525         throw invalid_attachment("attach: that execution is attached already");
00526     }
00527
00528     // At most one attacher each makes the graph a forest, so a cycle closes exactly when other is
00529     // somewhere up this chain. Walking it is the whole check.
00530     for (auto link = attachment_lifetime_->attacher.lock(); link; link = link->attacher.lock()) {
00531         if (link == other->attachment_lifetime_) {
00532             throw invalid_attachment("attach: the attachment would close a cycle");
00533         }
00534     }
00535
00536     if (!actuator_execute_.is_connected()) {
00537         actuator_execute_ = untangle::connect(other->action_execute);
00538     } else {
00539         actuator_execute_.add(&other->action_execute);
00540     }
00541
00542     if (!actuator_stop_.is_connected()) {
00543         actuator_stop_ = untangle::connect(other->action_stop);
00544     } else {
00545         actuator_stop_.add(&other->action_stop);
00546     }
00547
00548     // What ~execution() needs to take itself back out of these actuators: the link proves this
00549     // attacher is still there, and the two pointers are only ever followed while it does.
00550     other->attachment_lifetime_->attacher = attachment_lifetime_;
00551     other->attacher_execute_ = &actuator_execute_;
00552     other->attacher_stop_ = &actuator_stop_;
00553 }
00554
00566 template <typename otherT>
00567 bool detach(otherT& other) {
00568     // The actuator's own action list is the record of what is attached; nothing else has to keep
00569     // one. An action is stored as a pointer, so the attachment is found by identity.
00570     const auto& actions = actuator_execute_.actions;
00571     if (std::find(actions.begin(), actions.end(), &other.action_execute) == actions.end()) {
00572         return false;
00573     }
00574
00575     actuator_execute_.remove(&other.action_execute);
00576     actuator_stop_.remove(&other.action_stop);
00577
00578     other.attachment_lifetime_->attacher.reset();
00579     other.attacher_execute_ = nullptr;
00580     other.attacher_stop_ = nullptr;

```

```

00581
00582     return true;
00583 }
00584
00596 auto results() const {
00597     std::lock_guard<std::mutex> lock(results_mutex_);
00598     return results_;
00599 }
00600
00601 static constexpr auto default_name = "default_name";
00602
00608 std::function<void(void)> action_execute;
00609
00617 std::function<void(void)> action_stop;
00618
00627 std::function<bool(void)> action_is_running;
00628
00634 std::function<void(void)> on_finished;
00635
00655 std::function<void(std::exception_ptr)> on_error;
00656
00657 std::string name = default_name;
00658
00659 private:
00660 using queued_action_t = std::function<typename actionT::result_type(void)>;
00661
00666 void remember_poll(execution_poll& poll) {
00667     std::lock_guard<std::mutex> lock(polls_mutex_);
00668     polls_.push_back(&poll);
00669 }
00670
00671 void forget_poll(execution_poll& poll) {
00672     std::lock_guard<std::mutex> lock(polls_mutex_);
00673     std::erase(polls_, &poll);
00674 }
00675
00680 bool add_queued_action(queued_action_t action) {
00681     {
00682         std::lock_guard<std::mutex> lock(action_mutex_);
00683
00684         // Once stopped, the worker is on its way out and would never reach this action; dropping it
00685         // here is what keeps it from sitting in the queue looking as though it were pending.
00686         if (stopped_) {
00687             std::println(stderr, "warning: execution '{}' is stopped, action not added", name);
00688             return false;
00689         }
00690
00691         action_queue_.push_back(std::move(action));
00692     }
00693
00694     action_cv_.notify_one();
00695     return true;
00696 }
00697
00707 void execute_action(queued_action_t& action) {
00708     if constexpr (std::is_void_v<typename actionT::result_type>) {
00709         action();
00710     } else {
00711         auto value = action();
00712
00713         if (collecting_results_) {
00714             std::lock_guard<std::mutex> lock(results_mutex_);
00715             results_.push_back(std::move(value));
00716         }
00717     }
00718 }
00719
00726 void execute_actions() {
00727     actions_run_ = 0;
00728
00729     // The action is taken off the queue under the lock and invoked with the lock released: an
00730     // action is caller code that may run for a while, and may itself call add_action().
00731     for (;;) {
00732         queued_action_t action;
00733
00734         {
00735             std::lock_guard<std::mutex> lock(action_mutex_);
00736             if (action_queue_.empty()) {
00737                 break;
00738             }
00739
00740             action = std::move(action_queue_.front());
00741             action_queue_.pop_front();
00742
00743             // Under the lock that emptied the queue, so is_busy() sees the pop and this together or

```

```

00754     // neither.
00755     executing_action_ = true;
00756 }
00757
00758 // Nothing may leave this loop: an exception escaping a detached thread function calls
00759 // std::terminate.
00760 try {
00761     execute_action(action);
00762 } catch (const invalid_action& ia) {
00763     if (!report_error()) {
00764         std::println(stderr, "warning: execution '{}' dropped an invalid action: {}", name,
00765             ia.what());
00766     }
00767 } catch (const std::exception& e) {
00768     if (!report_error()) {
00769         std::println(stderr, "warning: execution '{}' dropped an action that threw: {}", name,
00770             e.what());
00771     }
00772 } catch (...) {
00773     if (!report_error()) {
00774         std::println(
00775             stderr, "warning: execution '{}' dropped an action that threw an unknown type", name);
00776     }
00777 }
00778
00779 executing_action_ = false;
00780
00781 // Counted whether it ran cleanly, reported a dead binding, or threw: it came off the queue,
00782 // and the queue is what the notification is about.
00783 ++actions_run_;
00784
00785 // The notification is raised without the lock: on_finished may call add_action(), which
00786 // takes it.
00787 bool drained = false;
00788 {
00789     std::lock_guard<std::mutex> lock(action_mutex_);
00790     drained = action_queue_.empty();
00791 }
00792
00793 if (drained) {
00794     notify_finished();
00795 }
00796 }
00797
00798 // Last, and part of the pass: an attachment is only ever reached through action_execute, which
00799 // is this function, so whoever drives this execution drives the ones attached to it.
00800 if (actuator_execute_.is_connected()) {
00801     actuator_execute_();
00802 }
00803 }
00804
00814 bool report_error() {
00815     if (!on_error) {
00816         return false;
00817     }
00818
00819     // The handler is the caller's code, running on a detached thread inside the try that caught the
00820     // action. Anything escaping it would leave the thread function and call std::terminate, so it
00821     // is caught here - a caller's broken handler costs its own warning, not the process.
00822     try {
00823         on_error(std::current_exception());
00824     } catch (...) {
00825         std::println(stderr, "warning: execution '{}' had on_error throw, and dropped what it threw",
00826             name);
00827     }
00828
00829     return true;
00830 }
00831
00836 void notify_finished() {
00837     if (actions_run_ == 0 || !on_finished) {
00838         return;
00839     }
00840
00841     {
00842         // An action may queue another, so a pass that drained can leave more behind it. That is the
00843         // next batch, not the end of this one.
00844         std::lock_guard<std::mutex> lock(action_mutex_);
00845         if (!action_queue_.empty()) {
00846             return;
00847         }
00848     }
00849
00850     on_finished();
00851 }
00852
00860 void wait_thread_to_finish() {

```

```

00861     auto resident = false;
00862     while (!running_.compare_exchange_weak(resident, true)) {
00863         resident = false;
00864         std::this_thread::sleep_for(std::chrono::milliseconds(1));
00865     }
00866 }
00867
00868 void execute() {
00869     // The results belong to this run, not to this run and the last one.
00870     {
00871         std::lock_guard<std::mutex> lock(results_mutex_);
00872         results_.clear();
00873     }
00874
00875     execute_actions();
00876
00877     std::println("execution '{}'' finishing thread", name);
00878
00879     // Must stay last: ~execution() may free this object the moment it reads false.
00880     running_ = false;
00881 }
00882
00883 void loop() {
00884     for (;;) {
00885         {
00886             std::unique_lock<std::mutex> lock(action_mutex_);
00887
00888             // Bounded: an attached execution has its own list and cannot notify this condition
00889             // variable.
00890             action_cv_.wait_for(lock, std::chrono::milliseconds(10),
00891                 [this] { return !action_queue_.empty() || !started_; });
00892
00893             if (!started_ && action_queue_.empty()) {
00894                 break;
00895             }
00896         }
00897         execute_actions();
00898     }
00899
00900     // A last pass for the attachments; this execution's own list is already empty here.
00901     execute_actions();
00902
00903     std::println("execution '{}'' thread finished", name);
00904
00905     // Must stay last: ~execution() may free this object the moment it reads false.
00906     running_ = false;
00907 }
00908
00909 std::thread thread_;
00910
00911 std::atomic_bool started_ = {false};
00912 std::atomic_bool running_ = {false};
00913
00914 // Set by stop(), and the difference between "not started yet" and "finished for good": actions
00915 // may be queued before run()/start(), but not after stop().
00916 std::atomic_bool stopped_ = {false};
00917
00918 // action_queue_, started_ and stopped_ are written by every thread that calls add_action() or
00919 // stop() and read by the worker; nothing touches them outside this mutex.
00920 mutable std::mutex action_mutex_;
00921 std::condition_variable action_cv_;
00922
00923 std::deque<queued_action_t> action_queue_;
00924
00932 using void_actuator = actuator<std::function<void(void)>>;
00933
00934 void_actuator actuator_execute_;
00935 void_actuator actuator_stop_;
00936
00948 std::shared_ptr<attachment> attachment_lifetime_ = std::make_shared<attachment>();
00949
00961 void_actuator* attacher_execute_ = nullptr;
00962 void_actuator* attacher_stop_ = nullptr;
00963
00964 // std::vector cannot hold void type; use an arbitrary type e.g. int
00965 using resultT = std::conditional<std::is_void<typename actionT::result_type>::value, int,
00966     typename actionT::result_type>;
00967
00971 std::vector<typename resultT::type> results_;
00972
00976 mutable std::mutex results_mutex_;
00977
00984 std::atomic_bool collecting_results_ = {false};
00985
00993 std::atomic_bool executing_action_ = {false};
00994
00996 std::vector<execution_poll*> polls_;

```

```
00997 mutable std::mutex polls_mutex_;
00998
01006 std::atomic_size_t actions_run_ = {0};
01007 };
01008 } // namespace async
01009 } // namespace untangle
```

Index

- ~execution
 - untangle::async::execution< actionT >, [20](#)
- ~execution_poll
 - untangle::async::execution_poll, [28](#)
- action_execute
 - untangle::async::execution< actionT >, [26](#)
- action_is_running
 - untangle::async::execution< actionT >, [26](#)
- action_stop
 - untangle::async::execution< actionT >, [26](#)
- add
 - untangle::async::execution_poll, [28](#)
- add_action
 - untangle::async::execution< actionT >, [23](#)
- async, [1](#)
- attach
 - untangle::async::execution< actionT >, [24](#)
- bind_action_and_function
 - untangle::async::execution< actionT >, [21](#)
- bind_action_and_method
 - untangle::async::execution< actionT >, [20](#)
- create_instance
 - untangle::async::execution< actionT >, [20](#)
- detach
 - untangle::async::execution< actionT >, [25](#)
- execution
 - untangle::async::execution< actionT >, [19](#)
- invalid_attachment
 - untangle::async::invalid_attachment, [30](#)
- is_busy
 - untangle::async::execution< actionT >, [22](#)
- is_running
 - untangle::async::execution< actionT >, [22](#)
 - untangle::async::execution_poll, [29](#)
- namespace untangle: functions, [13](#)
- on_error
 - untangle::async::execution< actionT >, [26](#)
- on_finished
 - untangle::async::execution< actionT >, [26](#)
- remove
 - untangle::async::execution_poll, [28](#)
- results
 - untangle::async::execution< actionT >, [25](#)
- run
 - untangle::async::execution< actionT >, [22](#)
- start
 - untangle::async::execution< actionT >, [23](#)
- stop
 - untangle::async::execution< actionT >, [23](#)
- untangle, [15](#)
- untangle::async::attachment, [17](#)
- untangle::async::execution< actionT >, [17](#)
 - ~execution, [20](#)
 - action_execute, [26](#)
 - action_is_running, [26](#)
 - action_stop, [26](#)
 - add_action, [23](#)
 - attach, [24](#)
 - bind_action_and_function, [21](#)
 - bind_action_and_method, [20](#)
 - create_instance, [20](#)
 - detach, [25](#)
 - execution, [19](#)
 - is_busy, [22](#)
 - is_running, [22](#)
 - on_error, [26](#)
 - on_finished, [26](#)
 - results, [25](#)
 - run, [22](#)
 - start, [23](#)
 - stop, [23](#)
- untangle::async::execution_poll, [27](#)
 - ~execution_poll, [28](#)
 - add, [28](#)
 - is_running, [29](#)
 - remove, [28](#)
 - withdraw, [29](#)
- untangle::async::invalid_attachment, [29](#)
 - invalid_attachment, [30](#)
 - what, [30](#)
- what
 - untangle::async::invalid_attachment, [30](#)
- withdraw
 - untangle::async::execution_poll, [29](#)